

Teoría 1 de Algorísmica Avanzada

Complejidad y fundamentos de grafos

Esta guía explica los conceptos de la primera clase y cómo aplicarlos a preguntas de teoría y análisis de código. La idea central es que el coste de un algoritmo depende de cuántas operaciones realiza al crecer la entrada y, en los grafos, también de cómo se guardan las relaciones entre los vértices.

Al terminar deberías poder justificar una complejidad, distinguir tipos de grafos, construir sus representaciones y seguir un recorrido DFS o BFS. Los ejemplos resueltos enseñan el razonamiento que conviene escribir en un examen, además del resultado.

Ruta de estudio

Bloque	Páginas	Qué aprender
Complejidad	2 a 9	Notación, bucles, operaciones de Python y recursión
Conceptos de grafos	10 a 12	Vértices, aristas, grados, caminos y problemas
Representaciones	13 a 15	Lista de aristas, matriz y listas de adyacencia
Recorridos	16 a 18	DFS, BFS y justificación de sus costes
Práctica y repaso	19 a 24	Ejercicios, soluciones y recordatorios

Cómo estudiar. Lee primero las explicaciones y rehace los ejemplos sin mirar la solución. Para cada código, escribe el tamaño de entrada, el número de iteraciones, el coste del cuerpo y el espacio auxiliar. Para cada grafo, fija antes si es dirigido y si tiene pesos.

Base de la explicación

Las referencias [A] indican páginas de AA01.pdf. Las referencias [G] corresponden al libro de grafos de la carpeta Libros. Al final se identifican los archivos y las secciones consultadas.

Los patrones adicionales de código se señalan como refuerzo. Sirven para practicar las ideas de la clase; no implican que esas preguntas vayan a aparecer en el examen.

La clase introduce algunos problemas y técnicas que se desarrollarán después. Aquí se explica qué significan, sin convertir esta guía en un desarrollo de todos esos algoritmos.

1 Qué mide la complejidad

La complejidad temporal describe cómo aumenta el trabajo de un algoritmo cuando crece la entrada. Medir segundos resulta útil para experimentar, pero mezcla el algoritmo con la máquina, el lenguaje, la implementación y la carga del sistema. El análisis teórico cuenta operaciones bajo un modelo explícito.

Elige bien el tamaño. Si recibes una lista c , normalmente $n = \text{len}(c)$. Si hay dos listas independientes, utiliza n y m . En un grafo, usa $v = |V|$ para los vértices y $e = |E|$ para las aristas. Escribir $O(n)$ sin definir n puede ocultar justamente lo que hay que explicar.

El modelo de coste

En el modelo habitual de estos ejercicios, una asignación, un acceso por índice, una comparación y una operación aritmética sobre valores de tamaño acotado cuestan $\Theta(1)$. Esta hipótesis permite concentrarse en el número de veces que se ejecutan.

No toda línea tiene coste constante. $\max(c)$ o $\text{sum}(c)$ recorren la lista. Una llamada a una función debe analizarse por el trabajo que realiza, aunque ocupe una sola línea. Si una operación aparece dentro de un bucle, su coste se paga en cada iteración.

Matiz sobre los números. Python permite enteros de tamaño arbitrario. Cuando el problema trata de números con muchos dígitos, sumar o multiplicar ya no es necesariamente constante. En los ejemplos de esta guía se utiliza el modelo habitual de coste unitario, salvo que se indique lo contrario.

Tiempo y espacio son preguntas diferentes

La complejidad espacial mide la memoria usada al crecer la entrada. Conviene distinguir memoria total, memoria de la salida y **espacio auxiliar**, que es la memoria adicional a la entrada.

Recorrer una lista con un acumulador cuesta $\Theta(n)$ tiempo y $\Theta(1)$ espacio auxiliar. Crear una copia cuesta $\Theta(n)$ tiempo y $\Theta(n)$ espacio adicional. La entrada sigue ocupando $\Theta(n)$, aunque el espacio auxiliar del primer algoritmo sea constante.

Mejor caso y peor caso

En una búsqueda lineal con salida inmediata, el mejor caso ocurre si el elemento está al principio: $\Theta(1)$. El peor ocurre si está al final o no aparece: $\Theta(n)$. El caso medio necesita una distribución de entradas; no se obtiene simplemente haciendo la media entre esos dos casos.

En el examen. Escribe «peor caso $\Theta(n)$ » si ese es el caso que has estudiado. Si el enunciado no lo especifica, analiza el peor caso y deja constancia de la elección. Un algoritmo puede tener el mismo orden en todos los casos.

Referencia: [A, pp. 7 a 10]; parámetros de grafos: [G, p. PDF 3].

2 Notación O Omega y Theta

La notación asintótica compara funciones cuando el tamaño de la entrada se hace suficientemente grande. No intenta dar un tiempo exacto ni conserva todas las constantes del recuento.

Notación	Significado para una función $T(n)$
$O(f(n))$	Cota superior a partir de cierto tamaño
$\Omega(f(n))$	Cota inferior a partir de cierto tamaño
$\Theta(f(n))$	Cota ajustada por arriba y por abajo

Definición de O. Existen constantes positivas c y n_0 tales que $T(n) \leq c \cdot f(n)$ para todo $n \geq n_0$. Para Ω se exige $T(n) \geq c \cdot f(n)$. Para Θ existen c_1 , c_2 y n_0 con $c_1 \cdot f(n) \leq T(n) \leq c_2 \cdot f(n)$ a partir de n_0 .

Por ejemplo, si $T(n) = 3n^2 + 5n + 7$, entonces $T(n) = \Theta(n^2)$. El término cuadrático domina a los demás. La función también es $O(n^3)$, pero esa cota es menos informativa. Si puedes justificar el crecimiento exacto salvo constantes, utiliza Θ .

Cómo simplificar correctamente

Suma de costes. En $n^2 + n + 10$ domina n^2 . En $n + \log n$ domina n . En $4n + 8$ queda orden lineal. Los factores constantes positivos no cambian la clase de crecimiento.

Producto de costes. Si realizas n veces un trabajo que cuesta $\Theta(n)$, el total es $\Theta(n^2)$. Si realizas n veces un trabajo de $\Theta(\log n)$, queda $\Theta(n \log n)$. La multiplicación debe corresponder al trabajo real del código.

Variables independientes. $n + m$ no se reduce a n salvo que conozcas una relación entre los tamaños. Lo mismo sucede con $v + e$: si el grafo tiene vértices aislados, e puede ser cero y aun así hay que procesar v vértices.

Tres confusiones que cuestan puntos

O no significa automáticamente peor caso. Puedes dar una cota O para el mejor, el medio o el peor caso. Son dos decisiones distintas: qué función de coste analizas y cómo acotas su crecimiento. La diapositiva 8 presenta O asociada al peor caso; esa es una aplicación habitual, no su definición matemática.

Ω no significa automáticamente mejor caso. Una cota inferior también puede describir la función de peor caso. Por ejemplo, el peor caso de un recorrido completo puede ser a la vez $O(n)$ y $\Omega(n)$, y por eso $\Theta(n)$.

$O(n)$ y $O(2n)$ son la misma clase. El factor 2 se absorbe en la constante. Sin embargo, $O(2^n)$ es exponencial: n está en el exponente y no se puede eliminar.

Referencia: [A, pp. 8 a 11].

3 Comparar órdenes de crecimiento

Para tamaños grandes, el orden habitual de las funciones de esta tabla es creciente. «Mejor» significa aquí menor crecimiento del coste, bajo los mismos supuestos y para tareas comparables.

Orden	Patrón habitual	Si duplicas n
$\Theta(1)$	Acceso por índice	Coste semejante
$\Theta(\log n)$	Reducir a la mitad en cada paso	Añades un número constante de pasos
$\Theta(n)$	Una pasada completa	Aproximadamente el doble
$\Theta(n \log n)$	n trabajos logarítmicos	Algo más del doble
$\Theta(n^2)$	Todos los pares	Aproximadamente cuatro veces
$\Theta(n^3)$	Todos los triples	Aproximadamente ocho veces
$\Theta(2^n)$	Todas las elecciones binarias	El coste se eleva al cuadrado
$\Theta(n!)$	Todas las permutaciones	Crecimiento aún mayor

La base de un logaritmo fijo mayor que 1 solo cambia un factor constante: $\log_2 n$ y $\log_{10} n$ tienen el mismo orden. La base de una exponencial sí importa: 2^n y 3^n no son de la misma clase Θ .

Las comparaciones de la diapositiva 11

La cadena $n, n \log n, n^2, n!$ está bien ordenada por crecimiento asintótico. La afirmación $O(n) < O(2n)$ es incorrecta como comparación de clases, porque ambas coinciden. También es incorrecto situar 2^n por debajo de n^2 para entradas suficientemente grandes.

Sobre $n!$ frente a 2^n , **$n!$ crece más deprisa asintóticamente**. Para comparar tiempos de dos programas con un n concreto necesitas además las constantes, la implementación y que las cotas describan ajustadamente sus costes. Que un programa sea $O(n!)$ por sí solo no demuestra que tarde más que otro $O(2^n)$, porque O puede ser una cota poco ajustada.

El ejemplo de las contraseñas

Con un alfabeto de 26 símbolos y longitud n hay 26^n cadenas. Si se prueban todas, el trabajo es exponencial. Cada carácter adicional multiplica el número de candidatos por 26.

Para $n = 8$: $26^8 = 208\,827\,064\,576$ candidatos. A un millón de pruebas por segundo, una exploración completa tarda unas 58 horas. Para $n = 12$, tarda unos 3 026 años, usando años de 365 días.

Esos tiempos corresponden a recorrer todo el espacio de búsqueda; encontrar una contraseña concreta podría detener el proceso antes. Son una ilustración matemática con una tasa fija de pruebas, no una predicción universal sobre sistemas reales.

Referencias: [A, pp. 11 a 13].

4 Resolver el código de la clase

Las páginas 6, 9 y 10 muestran el siguiente ejemplo. Para evitar confundir la lista con su longitud, llamaremos $n = \text{len}(c)$. Suponemos una lista no vacía de números y coste unitario para la suma y las comparaciones.

```
def main(c):
    a = 1
    b = 4
    d = max(c)
    for i in c:
        a = a + b
    return a
```

Fragmento	Trabajo que realiza	Coste
<code>a = 1 y b = 4</code>	Dos asignaciones	$\Theta(1)$
<code>d = max(c)</code>	Examina los n elementos	$\Theta(n)$
<code>for i in c</code>	n iteraciones de coste constante	$\Theta(n)$
<code>return a</code>	Devuelve el acumulador	$\Theta(1)$

Los bloques se ejecutan uno después de otro. Sus costes **se suman**: $T(n) = \Theta(1) + \Theta(n) + \Theta(n) + \Theta(1) = \Theta(n)$. No es $\Theta(n^2)$, porque `max(c)` está fuera del bucle.

La diapositiva 10 usa el recuento simplificado $1 + 1 + n + n + 1 = 2n + 3$. Si se contaran de otra manera las comparaciones, las asignaciones o el control del bucle, variaría la expresión exacta. El orden lineal permanecería igual.

Espacio auxiliar. Se usan unas pocas variables y el iterador no copia la lista: $\Theta(1)$, bajo el mismo modelo. La memoria de la entrada no se cuenta como auxiliar.

Lo que debes advertir en un examen

La variable `d` no se usa después, pero `max(c)` sí se ejecuta en el código mostrado. No elimines su coste al analizar el programa tal como está escrito. Quitar esa línea no cambiaría el orden final, porque todavía queda el recorrido completo.

Los valores de `c` no cambian el número de iteraciones y no hay salida anticipada. Por eso el orden temporal es $\Theta(n)$ para cualquier lista válida de tamaño n . El valor que devuelve es $1 + 4n$: el cuerpo suma 4 una vez por elemento, aunque no utilice `i`.

Si `c` está vacía, `max(c)` produce una excepción. Para analizar el recorrido normal se debe indicar el supuesto de entrada no vacía. No confundas el comportamiento de una entrada inválida con el mejor caso del problema bajo sus precondiciones.

Respuesta modelo. «Sea n la longitud de `c`. `max(c)` cuesta $\Theta(n)$ y el bucle realiza n operaciones constantes. Como son secuenciales, el tiempo es $\Theta(n)$ y el espacio auxiliar $\Theta(1)$ ».

Referencia: [A, pp. 6, 9 y 10].

5 Método para analizar bucles

El objetivo es convertir el código en un recuento de trabajo. Este procedimiento también ayuda a detectar hipótesis que el enunciado no deja claras.

1. Define el tamaño o los tamaños de entrada.
2. Indica el caso analizado y las operaciones que consideras constantes.
3. Divide el código en bloques y calcula cuántas veces se ejecuta cada uno.
4. Multiplica el número de ejecuciones por el coste del cuerpo; si cambia, usa una suma.
5. Suma los bloques secuenciales y conserva los términos dominantes.
6. Calcula por separado el espacio auxiliar y justifica el resultado.

Bucles consecutivos

```
for x in a:           # a tiene n elementos
    procesar(x)       # coste constante
for y in b:           # b tiene m elementos
    procesar(y)
```

El tiempo es $\Theta(n + m)$. Si ambas listas tienen longitud n , es $\Theta(2n) = \Theta(n)$. Dos bucles consecutivos no producen por sí mismos un coste cuadrático.

Bucles anidados independientes

```
for i in range(n):
    for j in range(m):
        total += 1
```

El cuerpo se ejecuta $n \cdot m$ veces: $\Theta(nm)$. Solo pasa a $\Theta(n^2)$ si $m = n$, o si una relación adecuada entre ambos parámetros lo justifica.

Un número fijo de repeticiones

```
for i in range(n):
    for j in range(10):
        total += 1
```

Hay $10n$ ejecuciones: $\Theta(n)$. Lo importante es que 10 no depende del tamaño de entrada. Aunque el código tenga dos niveles de bucles, su crecimiento es lineal.

Condiciones y salidas anticipadas

Si cada iteración elige entre dos ramas constantes, sigue costando $\Theta(1)$. Si una rama ejecuta un trabajo de $\Theta(n)$, analiza cuántas veces se toma. Para el peor caso, busca una entrada que realmente permita ejecutar la rama cara tantas veces como afirmas.

Un `break` o `return` puede mejorar el mejor caso. No demuestra que el peor sea constante: quizá exista una entrada que obligue a agotar el recorrido.

Refuerzo aplicado a los conceptos de [A, pp. 7 a 10].

6 Bucles cuyos límites cambian

Cuando el coste del bucle interior depende del índice exterior, escribe el número total de iteraciones como una suma. No multipliques automáticamente el máximo por n si quieres obtener una cota ajustada.

El patrón triangular

```
for i in range(n):
    for j in range(i):
        total += 1
```

Las iteraciones interiores son $0, 1, 2, \dots, n - 1$. En total hay $n(n - 1)/2 = \Theta(n^2)$. Dividir por 2 no cambia el orden. Si el interior fuera `range(i, n)`, sumarías $n + (n - 1) + \dots + 1$ y el orden también sería cuadrático.

Crecimiento multiplicativo

```
i = 1
while i < n:
    i *= 2
```

Después de k iteraciones, $i = 2^k$. Termina cuando 2^k alcanza n , así que k es del orden de $\log_2 n$. El tiempo es $\Theta(\log n)$ para n creciente. Un bucle que divide repetidamente un valor positivo entre 2 tiene el mismo patrón.

Truco. Con incremento $i += 2$ sigues teniendo $\Theta(n)$ iteraciones. Con $i *= 2$ tienes $\Theta(\log n)$. Sumar y multiplicar modifican de forma distinta la distancia hasta el límite.

Dos bucles y un resultado lineal

```
i = 1
while i < n:
    for j in range(i):
        total += 1
    i *= 2
```

El trabajo total es $1 + 2 + 4 + \dots$ hasta una potencia menor que n . Esta suma geométrica es $\Theta(n)$, aunque el exterior tenga $\Theta(\log n)$ iteraciones. Cada vuelta interior tiene un coste distinto.

Un resultado $n \log n$ con límites dependientes

```
for i in range(1, n + 1):
    for j in range(0, n, i):
        total += 1
```

Para cada i hay aproximadamente n/i iteraciones. La suma es del orden de $n \cdot (1 + 1/2 + \dots + 1/n) = \Theta(n \log n)$. Los redondeos añaden como mucho un término lineal y no cambian el resultado.

Regla útil. Sumas aritméticas suelen producir n^2 ; duplicaciones o mitades suelen producir $\log n$; sumas de potencias de 2 hasta n producen n . Comprueba siempre el cuerpo y los límites concretos.

Refuerzo para practicar análisis de código.

7 Operaciones de Python que ocultan trabajo

La sintaxis breve no garantiza coste constante. Esta tabla se refiere a listas de Python y a conjuntos o diccionarios basados en tablas hash; los elementos y sus comparaciones se suponen de tamaño acotado.

Operación	Coste habitual	Motivo o condición
<code>len(a)</code> , <code>a[i]</code>	$\Theta(1)$	Longitud almacenada y acceso por índice
<code>max(a)</code> , <code>sum(a)</code>	$\Theta(n)$	Recorren los n elementos
<code>x in a</code> , si <code>a</code> es lista	$O(n)$ en peor caso	Búsqueda secuencial
<code>a.copy()</code> , <code>a[:]</code>	$\Theta(n)$	Copian n referencias
<code>a[i:j]</code>	$\Theta(k)$	Copia los k elementos del tramo
<code>a + b</code>	$\Theta(n + m)$	Construye una lista nueva
<code>a.append(x)</code>	$O(1)$ amortizado	Alguna ampliación puede costar $\Theta(n)$
<code>a.pop()</code> al final	$O(1)$ amortizado	No desplaza todos los elementos
<code>a.pop(0)</code> , <code>a.insert(0, x)</code>	$O(n)$	Desplazan elementos
<code>x in s</code> , si <code>s</code> es set	$O(1)$ esperado	Puede ser $O(n)$ en peor caso
<code>sorted(a)</code>	$O(n \log n)$ en peor caso	No supone coste constante

Amortizado y esperado no son sinónimos. El coste amortizado reparte el trabajo de una secuencia de operaciones: n inserciones al final cuestan $O(n)$ en total, aunque alguna sea cara. El coste esperado de una tabla hash depende de las hipótesis sobre el comportamiento del hashing. Ninguno significa que toda operación individual sea constante en su peor caso.

Una llamada dentro del bucle

```
for x in a:
    m = max(a)
```

Hay n llamadas, cada una $\Theta(n)$: el tiempo es $\Theta(n^2)$. Si el máximo no depende de la iteración, calcularlo una vez antes del bucle reduce este fragmento a $\Theta(n)$.

Construir una lista por concatenación

```
salida = []
for x in a:
    salida = salida + [x]
```

Se copian listas de tamaños $1, 2, \dots, n$: $\Theta(n^2)$ tiempo. Con `salida.append(x)` el coste total es $O(n)$ amortizado. En ambas versiones la salida final ocupa $\Theta(n)$; la memoria simultánea no es $\Theta(n^2)$.

Para los recorridos de grafos. Un set de visitados evita una búsqueda lineal de pertenencia. Para BFS, una cola deque permite extraer por el principio en $\Theta(1)$, mientras que `list.pop(0)` desplaza elementos.

Refuerzo de implementación. La idea de análisis amortizado aparece en [D, p. PDF 3].

8 Recursión y memoria

Una función recursiva no es exponencial solo por llamarse a sí misma. Debes contar las llamadas, el tamaño de cada subproblema y el trabajo que se hace fuera de ellas. Esta página es refuerzo para reconocer patrones y comprender el DFS de la clase.

Una llamada que reduce el tamaño en uno

```
def f(n):
    if n <= 0:
        return 0
    return 1 + f(n - 1)
```

$T(n) = T(n - 1) + \Theta(1)$, con $T(0) = \Theta(1)$. Hay $\Theta(n)$ llamadas: tiempo $\Theta(n)$. La profundidad también es $\Theta(n)$, por lo que la pila ocupa $\Theta(n)$ espacio auxiliar. No basta con contar las variables de una sola llamada.

Una llamada que reduce a la mitad

Si la función hace una llamada con $n // 2$ y trabajo constante fuera de ella, $T(n) = T(n/2) + \Theta(1)$: tiempo $\Theta(\log n)$ y profundidad $\Theta(\log n)$. Si antes de llamar recorre n elementos, cambia a $T(n) = T(n/2) + \Theta(n)$ y el tiempo total es $\Theta(n)$.

Dos llamadas de tamaño casi igual al original

```
def g(n):
    if n <= 0:
        return 1
    return g(n - 1) + g(n - 1)
```

$T(n) = 2T(n - 1) + \Theta(1)$. El árbol de llamadas tiene niveles de tamaños 1, 2, 4, ..., 2^n : tiempo $\Theta(2^n)$. Pero la ejecución normal hace las llamadas una tras otra; su profundidad máxima es $\Theta(n)$, por lo que la pila ocupa $\Theta(n)$, no $\Theta(2^n)$.

Dos llamadas con $n/2$ y trabajo constante fuera de ellas dan $T(n) = 2T(n/2) + \Theta(1) = \Theta(n)$. Si además hay $\Theta(n)$ trabajo por nivel, queda $\Theta(n \log n)$. No sustituyas $n - 1$ por $n/2$ al reconocer el patrón.

Repetir estados y recordar resultados

La versión recursiva ingenua de Fibonacci recalcula los mismos estados muchas veces. Guardar resultados puede evitar esa repetición, pero requiere memoria. En los grafos, el conjunto de visitados cumple una función relacionada: impide volver a expandir una y otra vez el mismo vértice.

Espacio máximo simultáneo. Suma los datos que coexisten y la pila activa. No sumes automáticamente toda la memoria asignada a lo largo del tiempo. Si cada llamada mantiene una copia nueva, esa memoria también debe contarse.

Referencias: recursión del DFS [A, pp. 72 y 73]; subproblemas repetidos [D, sección 15.1, pp. PDF 8 a 12].

9 Qué es un grafo

Un grafo es una estructura que representa entidades y relaciones. Se escribe $G = (V, E)$, donde V es el conjunto de vértices o nodos y E el conjunto de aristas o arcos. Puede tener vértices sin ninguna arista; la información de V no se puede reconstruir siempre solo a partir de E .

Frente a una estructura lineal, un vértice puede relacionarse con varios otros. El dibujo es una ayuda: las posiciones en el papel no forman parte del grafo salvo que el problema les dé significado.

Grafos no dirigidos y dirigidos

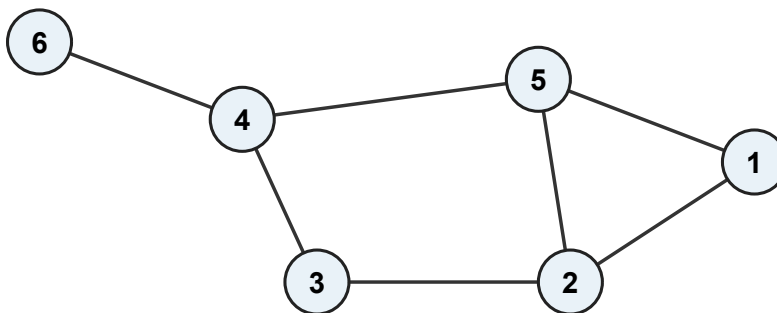
En un grafo **no dirigido**, la arista $\{u, w\}$ une ambos extremos sin orientación. $\{u, w\}$ y $\{w, u\}$ representan la misma arista. La amistad mutua es un ejemplo de modelado.

En un grafo **dirigido**, el arco (u, w) va de u a w . Puede existir sin que exista (w, u) . Enlaces de páginas web, relaciones de seguimiento o movimientos permitidos en un juego pueden modelarse así.

El ejemplo de seis vértices de la clase

$V = \{1, 2, 3, 4, 5, 6\}$

$E = \{\{1, 2\}, \{1, 5\}, \{2, 3\}, \{2, 5\}, \{3, 4\}, \{4, 5\}, \{4, 6\}\}$



Hay $v = 6$ y $e = 7$. Los vértices 1 y 2 son adyacentes porque comparten una arista; 1 y 4 no lo son, aunque puedes llegar de uno al otro siguiendo varias aristas. Este mismo ejemplo se usará para calcular grados y construir representaciones.

Pesos y modelado

Un grafo ponderado asigna a cada arista un peso: distancia, tiempo, coste o capacidad, según el problema. Ser ponderado es independiente de ser dirigido. El peso no es el grado del vértice ni el número de aristas.

Para modelar un problema, pregunta qué es un estado o entidad, qué relación crea una arista, si se puede recorrer en ambos sentidos y qué representa el peso. Dos modelos diferentes de la misma situación pueden plantear preguntas algorítmicas diferentes.

Referencias: [A, pp. 15 a 25 y 29 a 43].

10 Grafos simples y grados

Un **bucle** es una arista que conecta un vértice consigo mismo. Hay aristas paralelas cuando existen varias aristas entre los mismos extremos. Según la convención usada en AA01, un grafo simple es no dirigido, no tiene bucles y no tiene aristas paralelas.

También se habla de grafos dirigidos simples en otros textos, con una convención adaptada. En una pregunta de esta clase, utiliza la definición de las diapositivas y especifica el tipo de grafo.

Grado en un grafo no dirigido

El grado $d(u)$ cuenta las incidencias de aristas en u . En un grafo simple equivale al número de vecinos. Un vértice aislado tiene grado 0. Si se permiten bucles, cada bucle aporta **dos** al grado: sus dos extremos inciden en el mismo vértice.

Vértice del ejemplo	Vecinos	Grado
1	2 y 5	2
2	1, 3 y 5	3
3	2 y 4	2
4	3, 5 y 6	3
5	1, 2 y 4	3
6	4	1

La suma es $2 + 3 + 2 + 3 + 3 + 1 = 14 = 2e$. Esta identidad, $\sum d(u) = 2e$, se explica porque cada arista tiene dos extremos. Es una comprobación muy útil para detectar una arista olvidada o contada de más. Además, el número de vértices de grado impar siempre es par.

Grados en un grafo dirigido

El **grado de entrada**, $d^-(u)$, cuenta los arcos que llegan a u . El **grado de salida**, $d^+(u)$, cuenta los que salen. Un bucle dirigido aporta uno a cada uno.

Para los arcos $(1, 2)$, $(1, 5)$ y $(2, 5)$, el vértice 1 tiene salida 2 y entrada 0; el 2, salida 1 y entrada 1; el 5, salida 0 y entrada 2. Cada arco aporta exactamente una salida y una entrada al conjunto del grafo.

Por tanto, $\sum d^+(u) = e$ y $\sum d^-(u) = e$. Si se define grado total como entrada más salida, su suma es $2e$.

No confundas grado con peso. Un vértice unido por tres aristas de pesos 8, 2 y 100 tiene grado 3; la suma de pesos es otra magnitud. En listas de adyacencia de un grafo simple, la longitud de una lista de vecinos da el grado o el grado de salida.

Referencias: [A, pp. 22 a 27]; doble almacenamiento en listas [G, pp. PDF 5 y 6].

11 Caminos y preguntas sobre grafos

Una secuencia de vértices enlazados por aristas describe un recorrido. En un grafo dirigido debe respetar las flechas. Un camino simple no repite vértices; un ciclo simple vuelve al vértice inicial sin repetir los demás. La terminología sobre «camino» y «recorrido» puede variar, por lo que conviene explicitar las restricciones.

La **longitud** de un camino no ponderado es su número de aristas. Su coste ponderado es la suma de los pesos. Un camino con dos aristas de pesos 100 y 100 cuesta más que uno con tres aristas de peso 1.

Un grafo no dirigido es **conexo** si cualquier par de vértices está unido por un camino. Sus componentes conexas son los grupos máximos de vértices conectados entre sí. En grafos dirigidos se distingue conectividad fuerte, respetando las orientaciones, y débil, ignorándolas.

Qué pregunta cada problema de la clase

Alcanzabilidad. ¿Se puede llegar de s a t ? DFS o BFS permiten resolverla.

Camino más corto. ¿Qué camino minimiza la longitud o el coste entre s y t ? BFS sirve para el número de aristas en grafos sin pesos, o con un mismo peso positivo en todas las aristas. Los pesos generales requieren otros algoritmos.

Euler. Un recorrido euleriano utiliza cada **arista** exactamente una vez. Puede repetir vértices. Si empieza y termina en el mismo vértice, es un circuito euleriano. La condición esencial no es visitar cada vértice exactamente una vez.

Hamilton. Un camino hamiltoniano visita cada **vértice** exactamente una vez; un ciclo hamiltoniano vuelve después al inicial. Es un problema distinto de Euler. «Visitar todos los puntos y volver con mínimo coste» introduce la optimización del viajante de comercio.

Árbol de expansión mínimo o MST. En un grafo no dirigido, ponderado y conexo, conecta todos los vértices con un árbol de peso total mínimo. No busca una única ruta que visite todos los vértices ni minimiza necesariamente las distancias desde un origen.

Articulación y biconectividad. Un vértice de articulación aumenta el número de componentes conexas al eliminarlo junto con sus aristas. La biconectividad por vértices trata de evitar esos puntos únicos de fallo, con las convenciones pertinentes para grafos pequeños.

Planaridad. Un grafo es planar si admite un dibujo sin cruces entre aristas fuera de sus extremos comunes. Que un dibujo concreto tenga cruces no prueba que el grafo no sea planar.

Dificultad. Alcanzabilidad, caminos mínimos en sus variantes habituales, MST, Euler y planaridad admiten algoritmos polinómicos. Hamilton y el viajante general pertenecen a familias NP-completas y NP-difíciles, respectivamente; no se conoce un algoritmo polinómico general. Esta es una orientación, no una demostración de imposibilidad.

Referencia: [A, pp. 14, 35, 40 y 44].

12 Lista de aristas y matriz de adyacencia

La representación determina qué operaciones son rápidas y cuánto espacio necesitas. AA01 presenta tres posibilidades principales. Un array o una lista enlazada son contenedores posibles; lo decisivo es si almacenan todas las aristas juntas o los vecinos de cada vértice.

Lista de aristas

```
vertices = [1, 2, 3, 4, 5, 6]
aristas = [(1, 2), (1, 5), (2, 3), (2, 5),
           (3, 4), (4, 5), (4, 6)]
```

En el caso no dirigido, cada tupla representa una conexión sin orientación. Es suficiente guardar cada arista una vez. Con pesos, puedes usar tuplas (u, w, peso) . Mantén los vértices por separado para conservar también los aislados.

La lista es compacta y permite recorrer todas las aristas en $\Theta(e)$. Sin un índice adicional, comprobar si una arista existe o encontrar los vecinos de un vértice requiere $O(e)$ en el peor caso. Puede resultar útil para algoritmos que trabajan con las aristas como conjunto.

Matriz de adyacencia

Se numeran los vértices y se construye una matriz M de v filas y v columnas. En un grafo sin pesos, $M[u][w] = 1$ si existe la arista o arco correspondiente; en otro caso vale 0. En Python, adapta los índices si las etiquetas empiezan en 1.

M	1	2	3	4	5	6
1	0	1	0	0	1	0
2	1	0	1	0	1	0
3	0	1	0	1	0	0
4	0	0	1	0	1	1
5	1	1	0	1	0	0
6	0	0	0	1	0	0

En el ejemplo no dirigido, la matriz es simétrica: $M[u][w] = M[w][u]$. La diagonal es cero porque no hay bucles. En un grafo dirigido, la fila indica salidas y la columna entradas, y no hay obligación de simetría.

Costes. La matriz ocupa $\Theta(v^2)$, incluso con pocas aristas. Consultar una posición cuesta $\Theta(1)$, pero enumerar los vecinos de u exige revisar su fila: $\Theta(v)$. Inicializar toda la matriz también cuesta $\Theta(v^2)$.

Referencias: [A, pp. 61 a 64]; [G, sección 22.1, pp. PDF 4 a 6].

13 Listas de adyacencia y pesos

Una lista de adyacencia guarda, para cada vértice, los vecinos con los que tiene conexión. El conjunto completo incluye v contenedores, además de las referencias correspondientes a las aristas.

```
adj = {
    1: [2, 5],
    2: [1, 3, 5],
    3: [2, 4],
    4: [3, 5, 6],
    5: [1, 2, 4],
    6: [4]
}
```

En un grafo no dirigido, $\{u, w\}$ aparece como w en $\text{adj}[u]$ y u en $\text{adj}[w]$. No son dos aristas diferentes: son dos referencias a la misma conexión. La suma de longitudes es $2e$. En un grafo dirigido, normalmente se almacenan solo los vecinos de salida y la suma es e .

Espacio. $\Theta(v + e)$, incluyendo los contenedores de vértices aislados. Enumerar los vecinos de u cuesta $\Theta(d(u))$, o $\Theta(d^+(u))$ si es dirigido. Buscar uno concreto en una lista sin ordenar cuesta $O(d(u))$ en el peor caso. Guardarlos en conjuntos puede mejorar la búsqueda esperada, con otras hipótesis y costes.

Cómo guardar pesos sin ambigüedad

En listas puedes almacenar pares (vecino, peso): $\text{adj}[1] = [(2, 8), (3, 1)]$. El primer número identifica el destino; el segundo, el peso. Si el grafo es no dirigido, almacena también las entradas simétricas con el mismo peso.

En una matriz ponderada, la celda guarda el peso. Usa un indicador de ausencia que no se confunda con ningún peso válido, por ejemplo `None` o infinito cuando el algoritmo lo permita. Si existen aristas de peso cero, 0 no puede significar simultáneamente «no hay arista».

La diagonal cero de una matriz de distancias puede representar que quedarse en el mismo vértice cuesta 0. Eso no implica que haya un bucle de peso 0. Distingue una matriz de adyacencia de una matriz preparada para un algoritmo de distancias.

Una trampa al construir matrices en Python

```
# Correcto: cada fila es una lista diferente
M = [[0] * v for _ in range(v)]

# Incorrecto si después modificas celdas
M = [[0] * v] * v
```

En la segunda versión todas las filas referencian la misma lista. Cambiar una celda cambia aparentemente varias filas. Es un problema de corrección de la estructura, aunque la expresión parezca más corta.

Referencias: [A, pp. 64 a 66]; [G, sección 22.1, pp. PDF 5 a 7].

14 Elegir una representación

Esta comparación supone un grafo simple, ya construido, sin índices adicionales. $d(u)$ es el grado; en un grafo dirigido, utiliza el grado de salida para las listas. Las cotas de búsqueda son de peor caso.

Operación	Lista de aristas	Matriz	Listas de vecinos
Guardar el grafo completo	$\Theta(v + e)$	$\Theta(v^2)$	$\Theta(v + e)$
Consultar una arista	$O(e)$	$\Theta(1)$	$O(d(u))$
Enumerar vecinos de u	$\Theta(e)$	$\Theta(v)$	$\Theta(d(u))$
Recorrer todas las relaciones almacenadas	$\Theta(e)$	$\Theta(v^2)$	$\Theta(v + e)$
Añadir sin comprobar duplicados	$O(1)$ amortizado	$\Theta(1)$	$O(1)$ amortizado

La última fila supone vértices ya existentes, matriz ya reservada y uso de append en listas. Si debes comprobar que no existe la arista, añade el coste de buscarla. Añadir un vértice a una matriz puede requerir reconstruirla. Un contenedor de vértices no está incluido en la fila de recorrido de la lista de aristas, pero sí en la memoria del grafo completo.

Grafos dispersos y densos

Un grafo **disperso** o sparse tiene pocas aristas respecto de v^2 . En muchas familias dispersas $e = O(v)$, aunque la idea general es e mucho menor que v^2 . En un grafo denso, e tiene orden v^2 . Para un grafo simple no dirigido, el máximo es $v(v - 1)/2$; dirigido sin bucles, $v(v - 1)$.

Con $v = 10\,000$ y $e = 30\,000$, una matriz tiene 100 000 000 celdas. Las listas necesitan 10 000 contenedores y 60 000 entradas de vecinos si es no dirigido. Las cantidades de entradas muestran la diferencia, aunque los bytes reales dependen de la implementación.

Cómo justificar una elección

Si recorres vecinos y hay pocas aristas, las listas de adyacencia suelen ser adecuadas: evitan revisar conexiones inexistentes. Esto explica la elección habitual para grafos grandes y dispersos como muchas redes de enlaces.

Si consultas continuamente pares concretos, una matriz ofrece acceso directo. Puede ser razonable en grafos pequeños o densos. En un grafo denso, las listas también ocupan $\Theta(v^2)$; su ventaja de orden espacial desaparece.

Si procesas las aristas globalmente, una lista de aristas puede ser suficiente. Pregunta siempre qué operación domina el algoritmo, en vez de elegir solo por el nombre de la estructura.

Para el examen. «Elijo listas porque el grafo es disperso y el algoritmo enumera vecinos; su memoria es $\Theta(v + e)$ y cada enumeración cuesta $\Theta(d(u))$ ». La justificación conecta datos, operación y coste.

Referencias: [A, pp. 67 a 70]; [G, sección 22.1].

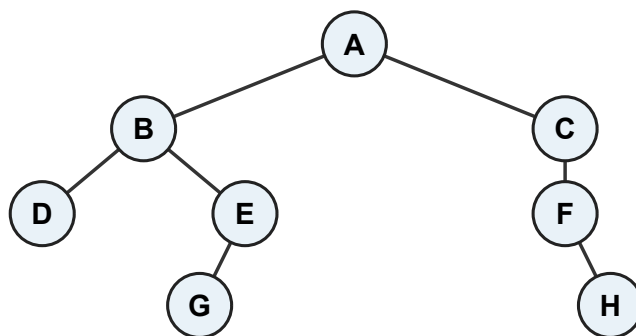
15 El DFS recursivo de la clase

Las páginas 72 y 73 representan un árbol con raíz A y el siguiente diccionario. Cada lista conserva un orden que determina cómo se visitan los hermanos.

```
graph = {
    'A': ['B', 'C'], 'B': ['D', 'E'],
    'C': ['F'],      'D': [],
    'E': ['G'],      'F': ['H'],
    'G': [],         'H': []
}
visited = set()

def explore(visited, graph, node):
    if node not in visited:
        print(node)
        visited.add(node)
        for neighbour in graph[node]:
            explore(visited, graph, neighbour)

explore(visited, graph, 'A')
```



La salida es **A, B, D, E, G, C, F, H**, un vértice por línea. Empieza en A, entra en B y profundiza hasta D. Como D no tiene hijos, regresa a B y explora E y G. Solo cuando termina toda esa rama vuelve a A para entrar en C, F y H.

Qué significa buscar en profundidad

DFS, depth first search, continúa por una rama mientras pueda. Al agotarla retrocede a una llamada anterior. La pila de llamadas recuerda qué vértices tienen vecinos pendientes. El código imprime al descubrir el vértice, así que en este árbol produce un **preorden**.

Si la impresión se colocara después del bucle de vecinos, obtendrías un orden de finalización o postorden: D, G, E, B, H, F, C, A. Recorrer por niveles es otro orden distinto, que corresponde a BFS.

El diccionario guarda hijos, no las conexiones inversas. El dibujo tiene apariencia de árbol no dirigido, pero esta estructura solo permite bajar desde A. Para representar todas las conexiones no dirigidas habría que guardar también los padres como vecinos.

Visitados se marca antes de profundizar. Esa decisión evita repetir la expansión de un vértice cuando existe un ciclo o cuando varios caminos llegan a él. La función puede recibir llamadas para vértices ya visitados; esas llamadas se descartan en la condición inicial.

Referencias: [A, pp. 71 a 73]; [G, sección 22.3, pp. PDF 18 y 19].

16 BFS y distancias por niveles

BFS, breadth first search, explora primero los vértices a una arista del origen, después los que están a dos, y así sucesivamente. Una cola FIFO guarda el orden pendiente: el primero que entra es el primero que sale.

En el árbol anterior, con los vecinos en el orden dado, BFS descubre **A, B, C, D, E, F, G, H**. Las distancias desde A son 0 para A; 1 para B y C; 2 para D, E y F; 3 para G y H.

Una versión que calcula distancias

```
from collections import deque

def bfs_distancias(adj, origen):
    dist = {origen: 0}
    cola = deque([origen])
    while cola:
        u = cola.popleft()
        for w in adj[u]:
            if w not in dist:
                dist[w] = dist[u] + 1
                cola.append(w)
    return dist
```

El diccionario `dist` sirve también para saber qué vértices se han descubierto. Se registra `w` **antes de encolarlo**, de modo que otro vecino no lo vuelva a introducir. Se supone que `adj` tiene una entrada para cada vértice y que las consultas al diccionario tienen coste esperado constante.

Por qué obtiene caminos mínimos sin pesos

Cuando se expande un vértice a distancia k , sus vecinos aún no descubiertos reciben $k + 1$. La cola procesa primero todos los candidatos de capas anteriores. Si hubiera un camino de menos aristas hasta w , w habría sido descubierto desde una capa anterior.

DFS puede llegar primero por un camino largo. Por ejemplo, con vecinos de s en el orden $[a, t]$ y aristas $s-a$, $a-b$, $b-t$, $s-t$, puede descubrir t después de tres pasos; BFS detecta que t está a una arista de s . Ambos encuentran alcanzabilidad, pero no ofrecen la misma garantía sobre el primer camino.

Amigos de amigos

La pregunta de la diapositiva 14 se interpreta como buscar personas cuya distancia mínima a Tom sea 2. BFS da precisamente esas capas en una red de amistad sin pesos. Excluye al propio Tom y a quienes ya son amigos directos, aunque también puedan aparecer mediante un recorrido de dos aristas.

Para limitar la exploración a distancia 2, basta con no expandir los vértices que ya están en esa capa. Encontrar un recorrido de dos pasos no equivale siempre a tener distancia mínima 2.

Con pesos diferentes, BFS minimiza el número de aristas, no la suma de pesos. No lo uses automáticamente para una ruta de kilómetros mínimos.

Referencias: [A, pp. 14 y 73]; [G, sección 22.2, pp. PDF 9 y 10].

17 Complejidad de los recorridos

El coste de DFS y BFS se entiende contando expansiones de vértices y revisiones de aristas. La presencia de bucles anidados o de recursión no determina por sí sola el resultado.

Con listas de adyacencia

Cada vértice se expande como máximo una vez gracias a visitados. Cada expansión recorre su lista de vecinos. En un grafo dirigido, la suma de longitudes es e ; en uno no dirigido, es $2e$. Por tanto, recorrer todo el grafo cuesta $\Theta(v + e)$, suponiendo comprobaciones de visitados de coste constante.

```
for u in adj:
    for w in adj[u]:
        procesar(u, w) # constante
```

Este fragmento tiene coste $\Theta(v + e)$, **no necesariamente $\Theta(v^2)$** . La suma de todas las iteraciones interiores es e o $2e$. Será cuadrático si el grafo es denso, porque entonces $e = \Theta(v^2)$.

En un árbol de v vértices hay $v - 1$ aristas: el recorrido cuesta $\Theta(v)$. En el código de la clase hay ocho vértices y siete relaciones de padre a hijo; la generalización del coste es lineal en el tamaño del árbol.

Desde un único origen

La versión que crea visitados solo para lo alcanzado explora el subgrafo alcanzable. Si tiene v_r vértices y e_r aristas o arcos relevantes, el coste es $\Theta(v_r + e_r)$, con las mismas hipótesis. Si el algoritmo inicializa arrays para todos los v vértices, esa preparación añade $\Theta(v)$.

Una sola llamada DFS no visita necesariamente todo un grafo desconectado. Para hacerlo, recorre los vértices y arranca una búsqueda cuando encuentres uno no visitado, compartiendo el conjunto de visitados. El coste global sigue siendo $\Theta(v + e)$.

Con otras representaciones

Con matriz de adyacencia, cada vértice expandido revisa una fila de v celdas. Un recorrido completo tiene coste $\Theta(v^2)$. Con una lista global de aristas, buscar vecinos mediante un barrido completo por cada vértice puede costar $\Theta(v \cdot e)$, además del trabajo de gestionar vértices.

Espacio auxiliar y trampas prácticas

DFS necesita visitados y una pila cuya profundidad puede llegar a v : $O(v)$ auxiliar. BFS necesita distancias o visitados y una cola que puede llegar a v : $O(v)$. La representación de entrada añade $\Theta(v + e)$ o $\Theta(v^2)$, pero se cuenta aparte.

Usar una lista para comprobar visitados puede introducir búsquedas de coste $O(v)$. Una cola con `pop(0)` puede añadir $\Theta(v^2)$ por desplazamientos. La recursión profunda en Python también puede superar el límite de llamadas, aunque el algoritmo tenga buen orden asintótico.

Referencias: [A, pp. 70 a 73]; [G, secciones 22.1 a 22.3].

18 Ejercicios de complejidad

Para cada fragmento indica el tiempo, el caso analizado y el espacio auxiliar. Supón $n = \text{len}(a)$, números de tamaño acotado, listas no vacías cuando se usa `max`, y que `total` ya está inicializado. Las soluciones están en la página siguiente.

Ejercicio 1 Recorridos consecutivos

```
x = max(a)
for y in a:
    total += y
```

Explica por qué dos recorridos de n elementos no producen n^2 .

Ejercicio 2 Trabajo oculto

```
for y in a:
    x = max(a)
```

Compara con el ejercicio anterior e indica una mejora válida.

Ejercicio 3 Pares sin repetir

```
for i in range(n):
    for j in range(i + 1, n):
        total += 1
```

Obtén el número exacto de ejecuciones del cuerpo y su orden.

Ejercicio 4 Un límite que se duplica

```
i = 1
while i < n:
    for j in range(n):
        total += 1
    i *= 2
```

No confundas este interior con `range(i)`.

Ejercicio 5 Encontrar un elemento

```
def contiene(a, objetivo):
    for x in a:
        if x == objetivo:
            return True
    return False
```

Determina mejor y peor caso para listas de longitud $n > 0$.

Ejercicio 6 Copiar tramos

```
for i in range(n):
    tramo = a[:i]
```

Calcula tiempo total y memoria máxima simultánea, sin contar la entrada.

19 Soluciones de complejidad

Ejercicio 1

$\max(a)$ cuesta $\Theta(n)$, y el bucle siguiente realiza n sumas constantes: $\Theta(n)$. Se suman porque son secuenciales. Resultado: **$\Theta(n)$ tiempo y $\Theta(1)$ auxiliar**, bajo el modelo indicado. El orden es el mismo para todas las entradas válidas de ese tamaño.

Ejercicio 2

Se ejecuta $\max(a)$ n veces y cada ejecución cuesta $\Theta(n)$. Resultado: **$\Theta(n^2)$ tiempo y $\Theta(1)$ auxiliar**. El máximo no cambia entre vueltas porque a no se modifica, así que puede calcularse una vez antes del bucle. En este fragmento, incluso se podría eliminar el bucle y conservar solo la asignación si no se requiere repetir ningún otro efecto.

Ejercicio 3

Para $i = 0$ hay $n - 1$ iteraciones; para $i = 1$, $n - 2$; la última vuelta tiene 0. El total es $(n - 1) + \dots + 1 + 0 = n(n - 1)/2$. Resultado: **$\Theta(n^2)$ tiempo y $\Theta(1)$ auxiliar**. Se están enumerando pares de índices con $i < j$.

Ejercicio 4

El exterior duplica i y da $\Theta(\log n)$ vueltas. En cada una, el interior realiza n iteraciones, independientemente de i . Resultado: **$\Theta(n \log n)$ tiempo y $\Theta(1)$ auxiliar**. Si el interior fuera $\text{range}(i)$, sumarías $1 + 2 + 4 + \dots$ y el resultado sería $\Theta(n)$.

Ejercicio 5

Mejor caso: el objetivo está en la primera posición, con **$\Theta(1)$ tiempo**. Peor caso: está al final o no aparece, con **$\Theta(n)$ tiempo**. El espacio auxiliar es $\Theta(1)$ en ambos casos. No afirmes una complejidad de caso medio sin describir cómo se distribuyen las entradas.

Ejercicio 6

Cada corte copia i referencias. El tiempo es $0 + 1 + 2 + \dots + (n - 1) = \Theta(n^2)$. El espacio auxiliar máximo es **$\Theta(n)$** porque no se conserva una colección con todos los tramos. Durante una reasignación pueden coexistir el tramo anterior y el nuevo; su tamaño total sigue siendo lineal.

Cómo escribir una justificación suficiente

Una buena respuesta dice qué se cuenta: «El bucle exterior realiza n iteraciones. En la vuelta i , el corte copia i elementos. La suma es $n(n - 1)/2$, por lo que el tiempo es $\Theta(n^2)$. Solo se mantienen tramos de tamaño $O(n)$, así que el espacio auxiliar es $\Theta(n)$ ».

Evita una explicación como «hay dos bucles, por tanto es cuadrático». Puede acertar por casualidad y falla en varios patrones de esta guía. El argumento correcto es el recuento de trabajo.

20 Ejercicios de grafos

Usa un grafo simple no dirigido con $V = \{A, B, C, D, E, F\}$ y $E = \{\{A, B\}, \{A, C\}, \{B, D\}, \{C, D\}, \{D, E\}\}$. F es aislado. Cuando recorras vecinos, usa orden alfabético.

Ejercicio 1 Propiedades

Calcula v y e , el grado de cada vértice y la suma de grados. Indica si es conexo, cuántas componentes tiene y si contiene un ciclo. Explica por qué una representación que solo guarde las aristas puede perder información.

Ejercicio 2 Representaciones

Construye las listas de adyacencia y la matriz en orden A, B, C, D, E, F. Cuenta las entradas de vecinos y las celdas de la matriz. Indica cuáles son cero por tratarse de un grafo simple y qué simetría esperas.

Ejercicio 3 Recorridos

Desde A, escribe el orden de descubrimiento de DFS recursivo con impresión antes de explorar vecinos. Después, escribe el orden de BFS y la distancia de cada vértice alcanzable. Indica qué ocurre con F.

Ejercicio 4 Analizar un recorrido

```
visitados = set()
def dfs(u):
    if u in visitados:
        return
    visitados.add(u)
    for w in adj[u]:
        dfs(w)

for u in adj:
    dfs(u)
```

¿Por qué el bucle exterior no hace que el tiempo pase a $\Theta(v \cdot (v + e))$? Supón consultas de coste esperado constante en el conjunto.

Ejercicio 5 Elegir representación

Un grafo tiene $v = 100\,000$ vértices y $e = 200\,000$ aristas, y el algoritmo necesita enumerar vecinos. Elige una representación y compara sus cantidades de almacenamiento. No hace falta estimar los bytes de objetos de Python.

Ejercicio 6 Distinguir los problemas

Un repartidor debe usar cada calle exactamente una vez. Otro problema pide visitar cada ciudad exactamente una vez. Un tercero quiere conectar todas las ciudades con cables de coste total mínimo. Identifica la familia de cada pregunta y explica qué elemento se usa o visita.

Las soluciones están en la página siguiente.

21 Soluciones de grafos

Ejercicio 1

$v = 6$ y $e = 5$. Los grados de A, B, C, D, E y F son **2, 2, 2, 3, 1 y 0**. Suman $10 = 2e$. Hay dos componentes: {A, B, C, D, E} y {F}. Existe el ciclo A-B-D-C-A. No es conexo, porque F no tiene camino a los demás. F no aparece en ninguna arista, por lo que E por sí solo no conserva su existencia.

Ejercicio 2

Las listas son A: [B, C]; B: [A, D]; C: [A, D]; D: [B, C, E]; E: [D]; F: []. Hay diez entradas de vecinos, porque cada arista se guarda en ambos extremos.

M	A	B	C	D	E	F
A	0	1	1	0	0	0
B	1	0	0	1	0	0
C	1	0	0	1	0	0
D	0	1	1	0	1	0
E	0	0	0	1	0	0
F	0	0	0	0	0	0

Hay 36 celdas. La diagonal es cero porque no hay bucles y la matriz es simétrica porque las conexiones son no dirigidas. La fila y columna de F son cero por estar aislado.

Ejercicio 3

DFS descubre **A, B, D, C, E**. Desde D entra primero en C y después en E; C ya estará visitado cuando vuelva a A. BFS descubre **A, B, C, D, E**, con distancias 0, 1, 1, 2 y 3, respectivamente. F no es alcanzable: no aparece en la búsqueda iniciada en A, o conserva distancia infinita si se inicializa para todos los vértices.

Ejercicio 4

El conjunto visitados es compartido por todas las llamadas. Cada vértice se expande una vez; cada lista se recorre una vez. Las llamadas exteriores sobre vértices ya visitados regresan inmediatamente. El coste global es $\Theta(v + e)$ bajo las hipótesis indicadas, y el espacio auxiliar $O(v)$. Reiniciar visitados para cada origen sería un algoritmo distinto.

Ejercicio 5

Las listas de adyacencia son adecuadas: e es pequeño respecto de v^2 y se enumeran vecinos. La matriz tendría **10 000 000 000 celdas**. Las listas tendrían 100 000 contenedores y 400 000 entradas de vecinos, suponiendo aristas no dirigidas. Su orden espacial es $\Theta(v + e)$.

Ejercicio 6

Usar cada calle corresponde a **Euler**, porque se usan aristas. Visitar cada ciudad corresponde a **Hamilton**, porque se visitan vértices. Conectar con cables de coste mínimo corresponde a **MST**, bajo las hipótesis de grafo no dirigido, ponderado y conexo. Volver al origen añade la condición de circuito o ciclo en los dos primeros problemas.

22 Estrategias que se presentan en la clase

La página 46 ofrece una vista general de técnicas que se trabajarán después. Interesa comprender su idea básica y qué hace falta justificar; su desarrollo completo no forma parte de esta primera introducción.

Algoritmos voraces o greedy

Un algoritmo voraz toma en cada paso la opción que parece mejor localmente y continúa sin reconsiderar las decisiones anteriores. Eso **no garantiza** una solución óptima global. Hay que demostrar que las elecciones son seguras para el problema concreto.

Por ejemplo, para devolver 6 unidades con monedas de valores 1, 3 y 4, elegir siempre la mayor moneda disponible produce $4 + 1 + 1$, tres monedas. La solución óptima es $3 + 3$, dos monedas. Es un contraejemplo a esa estrategia, no a todos los algoritmos voraces.

La clase menciona Kruskal y Prim para MST. Que sean voraces no basta para justificar su corrección: sus propiedades específicas permiten demostrarla. Tampoco debes trasladar una estrategia voraz válida para una variante de la mochila a otra sin comprobar sus condiciones.

Programación dinámica

Se divide el problema en estados o subproblemas y se guardan sus resultados para reutilizarlos. Resulta especialmente útil cuando hay **subproblemas solapados** y una relación que permite construir soluciones correctas a partir de estados menores. En problemas de optimización suele ser fundamental la subestructura óptima.

El número de estados y el trabajo por estado determinan el coste. Guardar resultados no convierte automáticamente cualquier problema en lineal. En el ejemplo del corte de barras del libro, evitar llamadas repetidas transforma una exploración exponencial en un cálculo cuadrático.

AA01 cita, entre otros, Floyd Warshall, mochila y Levenshtein como aplicaciones. Aquí basta reconocer la idea de reutilizar resultados; las recurrencias de esos algoritmos requieren su explicación posterior.

Enumeración y poda

Backtracking construye soluciones parciales, prueba decisiones y retrocede cuando una rama no puede llevar a una solución válida. Puede ahorrar mucho trabajo, pero en el peor caso seguir explorando un número exponencial de posibilidades.

Ramificación y poda, o branch and bound, se aplica a optimización: mantiene una solución factible y descarta ramas cuya cota demuestra que no pueden mejorarla. En minimización, una cota inferior de una rama al menos tan grande como el coste de la mejor solución conocida permite descartarla si solo buscamos mejorar ese coste.

Qué justificar. Explica qué información guarda cada técnica, por qué una decisión o poda es válida y cuánto trabajo puede quedar en el peor caso. Una idea que parece rápida necesita además un argumento de corrección.

Referencias: [A, p. 46]; [D, sección 15.1 y presentación de las técnicas de diseño].

23 Repaso para el examen y referencias

Análisis de código

Antes de calcular, define n o los parámetros necesarios, fija el caso y detecta llamadas que recorren datos. Después cuenta ejecuciones: bloques consecutivos se suman; trabajo repetido se acumula con productos o sumas, según cambie el coste.

Reconoce estos patrones. Una pasada: $\Theta(n)$. Pares: $\Theta(n^2)$. Duplicar o dividir por una constante: $\Theta(\log n)$. n tareas logarítmicas: $\Theta(n \log n)$. Copiar una lista de tamaño i para i de 1 a n : $\Theta(n^2)$. No olvides el coste de copiar, buscar y desplazar.

Da una cota ajustada cuando puedas. O es superior, Ω inferior y Θ ajustada. Ninguna de ellas sustituye a «mejor caso» o «peor caso». Los factores constantes se eliminan; los exponentes variables no.

Memoria. Cuenta datos adicionales y pila activa. Recuerda que tiempo exponencial puede coexistir con espacio lineal, y que memoria total y espacio auxiliar no son lo mismo.

Grafos

Define si hay dirección y pesos. No dirigido: $\sum d(u) = 2e$. Dirigido: $\sum d^+(u) = \sum d^-(u) = e$. Euler usa aristas; Hamilton visita vértices. El coste de una ruta ponderada suma pesos, mientras que la distancia sin pesos cuenta aristas.

Representación y recorrido. Matriz: $\Theta(v^2)$ espacio y consulta de arista $\Theta(1)$. Listas de adyacencia: $\Theta(v + e)$ espacio y recorrido de vecinos proporcional al grado. DFS y BFS completos con listas cuestan $\Theta(v + e)$; con matriz, $\Theta(v^2)$, bajo las hipótesis explicadas.

DFS profundiza con una pila; BFS avanza por niveles con una cola. El orden concreto depende del orden de vecinos. BFS calcula distancias mínimas en número de aristas; DFS no garantiza esa propiedad. Marca visitados antes de expandir o encolar y contempla los vértices aislados.

Referencias y lecturas de refuerzo

[A] Santi Seguí. AA01.pdf, curso 2026 a 2027. Archivo Semana 1/AA01.pdf. Complejidad: pp. 6 a 13. Conceptos, propiedades y aplicaciones de grafos: pp. 14 a 44. Técnicas de diseño: p. 46. Representaciones: pp. 59 a 70. Recorridos: pp. 71 a 73. Las páginas se cuentan desde el inicio del PDF.

[G] Grafos.pdf, carpeta Libros. Extracto de Introduction to Algorithms, Cormen, Leiserson, Rivest y Stein. Capítulo 22: sección 22.1 para representaciones (pp. PDF 4 a 8; impresas 589 a 593), 22.2 para BFS (pp. PDF 9 a 17; impresas 594 a 602) y 22.3 para DFS (desde p. PDF 18; impresa 603). Se han usado especialmente las explicaciones de almacenamiento y recuento de los recorridos.

[D] ProgramaciónDinámicaYAlgoritmosGreedy.pdf, carpeta Libros. Extracto del mismo libro. Introducción al análisis amortizado: p. PDF 3, impresa 358. Sección 15.1: ejemplo de llamadas repetidas y resultados guardados, pp. PDF 8 a 12, impresas 363 a 367. Sirve como refuerzo conceptual de las páginas 8 y 9 de esta guía.